30+ DevOps + AI Project Ideas

Beginner (10 projects)

1. Containerized ML Inference API

What: Build a Dockerized FastAPI/Flask app that serves a simple ML model (e.g., image classifier).
Tools: Python, FastAPI/Flask, Docker, GitHub Actions (CI), GitHub Pages or simple VPS.
Steps: train small model locally → wrap with FastAPI → containerize → add simple CI to run unit tests and build image → push to Docker Hub → deploy to VPS or cloud run.
Shareable assets: repo, Dockerfile, workflow YAML, demo GIF.
Resources: tutorial repos on model deployment and FastAPI (search "ml model deployment FastAPI Docker tutorial").

---

2. CI Pipeline for a Data Science Project (GitHub Actions)

What: Create CI that lints code, runs unit tests, and validates datasets (schema checks) on push.
Tools: GitHub Actions, pytest, pydantic or Great Expectations, pre-commit.
Steps: repo with data + code → write tests for data and functions → GitHub Actions workflow to run tests on PRs.

---

3. Model Versioning with DVC + Git

What: Put data and model versioning in place using DVC and Git to track experiments.
Tools: DVC, Git, remote storage (S3/GDrive), ML training script.
Steps: initialize DVC → track data/model artifacts → push to remote → demonstrate branching and reproducibility.

---

4. Automated Model Training on Schedule (Cron + CI)

What: Schedule a nightly job to retrain a small model using GitHub Actions or a cron runner.
Tools: GitHub Actions scheduled workflows, Docker, model training script.

Steps: create a scheduled workflow that triggers training, tests, and pushes the new model artifact to DVC or a model registry.

---

5. Simple Monitoring Dashboard for Model Metrics

What: Expose model inference metrics (latency, error rate) to Prometheus + Grafana.
Tools: Prometheus, Grafana, small exporter endpoint, Docker Compose.
Steps: instrument app → push metrics → Grafana dashboard with panels for latency/accuracy.

---

6. Serverless ML Inference (Cloud Run / Lambda)

What: Deploy a light ML model as serverless function (Google Cloud Run or AWS Lambda + API Gateway).
Tools: Cloud Run or AWS Lambda, Docker image or serverless function, SAM or Cloud Build.
Steps: package model with inference code → container / serverless handler → deploy and test.

---

7. Data Pipeline with Airflow (local) + ETL

What: Build a small ETL pipeline that fetches data, transforms, and stores it for training.
Tools: Apache Airflow (Docker), Python, SQLite/Postgres.
Steps: create DAGs for fetch-transform-load → run locally in Docker Compose → schedule daily runs.

---

8. Log-based Anomaly Detector (ELK + ML)

What: Ship logs to ELK stack, build an ML model that reads logs and flags anomalies.
Tools: Elastic Stack (Filebeat/Logstash/Elasticsearch/Kibana), Python model.
Steps: collect logs → store in Elasticsearch → export sample data → train anomaly detection → surface alerts in Kibana.

---

9. Simple A/B Test Automation for Model Versions

What: Router that splits traffic between two model versions and collects metrics.
Tools: NGINX or small proxy, FastAPI, lightweight metrics collection (Prometheus).

Steps: deploy v1 and v2 → proxy splits traffic → collect performance metrics → automatic rollouts based on metric thresholds.

---

10. Notebook-to-Pipeline Converter (basic)

What: Convert a Jupyter notebook into a script-based pipeline and add CI to test it.
Tools: nbconvert, GitHub Actions, tox/pytest.
Steps: take a trained-notebook → convert to modular scripts → add tests → CI validates runs.

Intermediate (13 projects)

11. End-to-end MLOps Pipeline (MLflow + Docker + Kubernetes)

What: Complete pipeline: experiment tracking (MLflow), containerized training, model registry, and deployment to Kubernetes.
Tools: MLflow, Docker, Kubernetes (minikube / kind), GitHub Actions, DVC.
Steps: track experiments with MLflow → register models → create Docker image for inference → deploy to Kubernetes → automated tests and rollout.
Resource examples: MLflow docs and community projects (see MLflow & MLOps repos).

---

12. GitOps for ML (ArgoCD + Kubeflow)

What: Use GitOps to manage deployments of Kubeflow pipelines & model-serving components.
Tools: ArgoCD, Kubeflow, GitHub, Helm charts.
Steps: store manifests in Git → ArgoCD watches repo → deploy/rollback Kubeflow components → integrate ML pipeline triggers.
Resource example: argocd-mlops-pipeline examples.(github.com)

---

13. MLOps with Kubeflow Pipelines + GitHub Actions

What: Build a Kubeflow pipeline that trains and evaluates on Kubernetes and tie it to CI.
Tools: Kubeflow Pipelines, GitHub Actions, Kubernetes.
Steps: define pipeline components → create CI to run unit/integration tests → run/publish pipeline runs.

---

14. Continuous Training & Deployment (CML / GitHub Actions)

What: Implement Continuous Machine Learning workflows that trigger retraining on new data and auto-deploy.
Tools: CML (Continuous Machine Learning), GitHub Actions, DVC, MLflow.
Steps: when new data is pushed → CI triggers training → evaluate → if metric improves, create PR with model and auto-merge.
Resource example: CML and actions-ml-cicd collections.(github.com)

---

15. End-to-end LLMOps: Deploy a Small LLM with Seldon/BentoML

What: Containerize and serve a small language model, add monitoring & scaling on Kubernetes.
Tools: Seldon Core or BentoML, Kubernetes, KEDA/HPA, Prometheus/Grafana.
Steps: package model with a server → Seldon/Bento wrapper → deploy to K8s → add autoscaling + monitoring.
Resource example: Seldon Core project.(github.com)

---

16. MLOps Infra as Code (Terraform + Cloud)

What: Use Terraform to provision repeatable infrastructure for ML workloads (storage, compute, ECR, EKS/GKE).
Tools: Terraform, AWS/GCP/Azure, modules for ECS/EKS/SageMaker.
Steps: write reusable Terraform modules → provision infra → deploy training & serving components.
Resource example: AWS MLOps Terraform template.(github.com)

---

17. Feature Store + Serving (Feast) Integration

What: Build a small feature store for online and offline features and use them in training and serving.
Tools: Feast, Postgres/Redis, Python feature pipelines, Kafka (optional).
Steps: define features → pipeline to compute/update → use in model training and serve low-latency features at inference.

---

18. Canary Deployments for ML Models with Argo Rollouts

What: Implement safe rollout strategies for model updates with metrics-driven promotion.
Tools: Argo Rollouts / Flagger, Kubernetes, Prometheus.
Steps: deploy baseline → configure rollout strategy → automated promotion/rollback based on metrics.

---

19. Secure MLOps: Secrets, RBAC, and Scanning

What: Add security best practices — secrets management, image scanning, and RBAC for MLOps pipelines.
Tools: HashiCorp Vault, Trivy, Gatekeepers (OPA), Kubernetes RBAC.
Steps: integrate Vault for secrets, add image scanning in CI, enforce policies via OPA/Gatekeeper.

---

20. Model Explainability + Monitoring (Evidently/AIX360)

What: Add explainability reports and data drift monitoring to an existing inference pipeline.
Tools: Evidently, AIX360, Prometheus, Grafana.
Steps: generate explainability reports → set alerts for drift → visual dashboard for stakeholders.

---

21. Real-time Inference with Kafka + Stream Processing

What: Build a streaming inference pipeline where data flows through Kafka, gets enriched by models, and stored.
Tools: Apache Kafka, Faust/ksqlDB, model-serving endpoint, Docker, Kubernetes.
Steps: produce messages → consumer calls model → publish results to downstream topic → store and monitor.

---

22. Build a Reproducible Benchmarking Suite for Models

What: Automated benchmarking across hardware/containers to compare model latency and throughput.
Tools: locust/hey, Kubernetes, Docker, CI to run benchmarks.
Steps: define tests, run on multiple instance types, create report that can be shared.

---

23. Data Lineage & Governance Demo

What: Show how lineage is tracked across datasets, transformations and model outputs.
Tools: OpenLineage, Data Catalog (e.g., Amundsen), Airflow.
Steps: instrument pipelines to record lineage → display in a catalog → connect to model artifacts.

Advanced (12 projects)

24. Full GitOps MLOps Platform (ArgoCD + Kubeflow + MLflow)

What: A self-hosted MLOps platform managed by GitOps with environment promotion and multi-tenant support.
Tools: ArgoCD, Kubeflow, MLflow, Kubernetes, Helm.
Steps: configure multi-environment repos → ArgoCD sync → RBAC → multi-tenant pipelines and model registries.

---

25. Scalable Training on Spot Instances + Checkpointing

What: Build distributed training workflows that use spot instances to reduce cost and save checkpoints.
Tools: Horovod/PyTorch Lightning, Kubernetes, cloud spot/scale sets, object storage.
Steps: containerize training → use orchestration to spin workers → checkpoint to object storage → resume on failure.

---

26. On-prem + Cloud Hybrid MLOps (Edge Inference)

What: Deploy model training in cloud, serve models on edge devices with syncing and version control.
Tools: K3s, Seldon/Bento, IoT device container runtime, CI for edge deployments.
Steps: train in cloud → package for edge → automated sync and rollout → monitor offline metrics.

---

27. LLMOps: Fine-tune, Serve, and Monitor a Custom LLM

What: Fine-tune an open LLM on domain data, serve with low-latency, and monitor hallucinations & safety.
Tools: Transformers, LoRA/QLoRA techniques, Seldon/Bento, Prometheus, guardrails libraries.
Steps: fine-tune small model → deploy via Seldon/Bento → add safety filters → collect metrics and user feedback.

---

28. Automated Compliance Pipeline (audit logs + immutable artifacts)

What: Build an auditable pipeline that records every code/data/model change for compliance needs.
Tools: WORM storage, Git commit signed artifacts, build provenance (in-toto), OpenSearch/Elastic for logs.
Steps: sign releases → store artifacts immutably → present audit reports and timelines.

---

29. Multi-model Serving Platform with Model Mesh

What: Architect and implement a Model Mesh to route requests to specialized models dynamically.
Tools: Seldon Core 2 (model mesh capabilities), service mesh (Istio), Kubernetes.
Steps: define model graph → implement routing rules → autoscale and monitor each model node.

---

30. AutoML + CI/CD for Model Selection

What: Integrate an AutoML process that runs trials and the CI automatically promotes the best model.
Tools: AutoGluon/Auto-sklearn/Vertex AutoML, CML, MLflow for tracking.
Steps: run AutoML searches → register candidates → CI evaluates and promotes winning model to staging.

---

31. Data Drift Remediation Automation

What: Detect drift and automatically trigger retraining or rollback to previous model versions.
Tools: Evidently/Prometheus, GitHub Actions/CML, MLflow, DVC.
Steps: monitor production data distributions → when drift detected → trigger retrain job → evaluate and deploy if better.

---

32. Cross-cloud Model Serving & Failover

What: Deploy a model serving layer across two clouds with traffic failover and synced registries.
Tools: Multi-cloud Kubernetes clusters, Seldon, Global load balancer, object storage replication.
Steps: replicate registry → configure global LB → perform failover drills and measure RTO/RPO.

---

33. ML Cost Optimization System

What: Track compute & storage costs for ML experiments and suggest cheaper configuration automatically.
Tools: Cloud billing APIs, MLflow, dashboards (Grafana), policy engine.
Steps: collect cost metrics per run → create cost-per-accuracy reports → CI alerts if runaway costs.

---

34. Build a Tiny MLOps SaaS Mock (multi-tenant)

What: Prototype a SaaS product that offers hosted model tracking & serving for multiple teams.
Tools: Django/Flask, Kubernetes, multi-tenant DB, MLflow or custom registry.
Steps: design tenant isolation → implement onboarding → billing telemetry & dashboards.

---

35. Advanced LLMOps: Retrieval-Augmented Generation (RAG) + Vector DB + Ops

What: Build a RAG pipeline: ingestion, vectorization, retrieval, and LLM serving with observability.
Tools: Milvus/Pinecone/Weaviate, LangChain, Seldon/Bento, Prometheus.
Steps: ingest docs → vectorize → build retriever → LLM prompt pipeline → deploy with monitoring and cost controls.